

Checklist:

Team Topologies – Four Fundamental Team Topologies

Organization-Level Checklist

- ☐ We have identified which teams are **stream-aligned**, **enabling**, **complicated-subsystem**, or **platform** teams.
- ☐ Most teams are **stream-aligned**.
- ☐ We do not have **vague “support,” “ops,” or “shared services”** teams without a clear topology.
- ☐ Each team has a clear purpose, ownership area, and interaction model.
- ☐ Team responsibilities reduce ambiguity rather than increase it.
- ☐ Teams are designed to improve **flow of change**, not just optimize resource utilization.
- ☐ Teams are loosely coupled and can make progress without excessive coordination.
- ☐ Hand-offs between teams are minimized.
- ☐ Team boundaries reduce cognitive load for stream-aligned teams.
- ☐ Platform, enabling, and complicated-subsystem teams exist to support stream-aligned teams, not control them.
- ☐ The ratio of stream-aligned teams to other team types is roughly between **6:1** and **9:1**.

Stream-Aligned Team Checklist

A stream-aligned team should own a continuous flow of work aligned to a business domain, product, service, user journey, or customer need.

- ☐ The team is aligned to a single valuable stream of work.
- ☐ The team owns delivery from idea to production.
- ☐ The team is close to customers or users.
- ☐ The team can deliver value quickly, safely, and independently.
- ☐ The team has minimal hand-offs to other teams.
- ☐ The team has long-term ownership of its area, not temporary project ownership.
- ☐ The team has the capabilities needed to build, run, and evolve its service.
- ☐ The team can monitor and operate its software in production.
- ☐ The team regularly evaluates flow, quality, and sustainability.
- ☐ The team has time to address technical debt and code quality.
- ☐ The team uses feedback and experimentation to improve.
- ☐ The team proactively reaches out to enabling, platform, or complicated-subsystem teams when needed.

Stream-Aligned Team Capabilities

Check whether the team has access to these capabilities, either internally or through lightweight support:

- ☐ Product ownership or product management
- ☐ Design and architecture
- ☐ Development and coding
- ☐ Testing and quality assurance
- ☐ User experience
- ☐ Application security
- ☐ Infrastructure and operability
- ☐ Metrics and monitoring
- ☐ Commercial or operational viability analysis

Important check:

- ☐ We do **not** assume each capability requires a separate specialist role.
- ☐ Specialists do not become bottlenecks for the team.
- ☐ Team members can collaborate across skill boundaries.

Enabling Team Checklist

An enabling team helps stream-aligned teams build capabilities, adopt new practices, and overcome knowledge gaps.

- ☐ The team is made up of specialists in a technical or product area.
 - ☐ The team helps stream-aligned teams learn, rather than taking over the work permanently.
 - ☐ The team's goal is to increase the autonomy of stream-aligned teams.
 - ☐ The team works with stream-aligned teams for a limited period when possible.
 - ☐ The team avoids becoming a permanent dependency.
 - ☐ The team actively understands the needs of stream-aligned teams.
 - ☐ The team establishes regular checkpoints with the teams it supports.
 - ☐ The team stays ahead of new tools, practices, and approaches.
 - ☐ The team shares both good news and bad news about technology choices.
 - ☐ The team promotes learning across teams.
 - ☐ The team acts more like a coach or consultant than a command-and-control authority.
 - ☐ The team avoids becoming an ivory tower.
-

Complicated-Subsystem Team Checklist

A complicated-subsystem team owns a part of the system that requires deep specialist knowledge.

- ☐ The subsystem genuinely requires specialist expertise.
- ☐ Most stream-aligned teams would struggle to understand or safely change the subsystem.
- ☐ The team reduces cognitive load for stream-aligned teams.

- ☐ The team exists because embedding the expertise everywhere would be impractical.
- ☐ The team owns a clearly bounded subsystem.
- ☐ The team collaborates closely with stream-aligned teams during early exploration.
- ☐ The team reduces interaction once the subsystem stabilizes.
- ☐ The team prioritizes work based on the needs of stream-aligned teams.
- ☐ The team delivers higher quality and speed than stream-aligned teams would.
- ☐ There are only a small number of complicated-subsystem teams.

Examples to check against:

- ☐ Video processing engine
- ☐ Mathematical model
- ☐ Financial reconciliation engine
- ☐ Face-recognition system
- ☐ Other highly specialized subsystems

Platform Team Checklist

A platform team provides internal services that help stream-aligned teams deliver with less cognitive load.

- ☐ The platform enables stream-aligned teams to deliver with greater autonomy.
- ☐ The platform is treated as an internal product.
- ☐ The platform provides self-service APIs, tools, services, documentation, or support.
- ☐ The platform reduces cognitive load for stream-aligned teams.
- ☐ The platform team understands its users' needs.
- ☐ The platform has clear user groups, such as developers, testers, product owners, and service engineers.
- ☐ The platform has defined hours of operation and support expectations.
- ☐ The platform is reliable, usable, and fit for purpose.
- ☐ The platform team uses product-management techniques.
- ☐ The platform team collects feedback from consuming teams.
- ☐ The platform team focuses on usability and developer experience.
- ☐ The platform avoids becoming too large or dominant.

- ☐ The platform is “just big enough” to help teams move faster.
- ☐ The platform provides a thin viable platform before expanding.
- ☐ The platform exposes useful constraints without slowing teams down.
- ☐ Platform adoption is understood as gradual, not immediate.

Platform Design Checklist

- ☐ The platform provides self-service capabilities.
- ☐ Teams can consume platform services without lengthy manual coordination.
- ☐ Documentation is clear, current, and task-focused.
- ☐ Onboarding a new developer to the platform is easy.
- ☐ The platform abstracts away infrastructure, networking, or cross-cutting concerns where useful.
- ☐ The platform does not hide unstable or poorly understood lower-level systems.
- ☐ Platform layers and dependencies are documented.
- ☐ The platform team understands what the platform depends on.
- ☐ The platform reduces, rather than increases, security and audit burden for stream-aligned teams.
- ☐ Internal pricing, usage tracking, or demand management is considered where demand is hard to control.

Avoiding Team Silos

- ☐ We avoid teams organized only by single functional expertise.
- ☐ Testing, QA, database, architecture, UX, and operations work are not isolated as mandatory hand-off stages.
- ☐ Cross-functional teams are preferred where possible.
- ☐ Specialist teams provide services or enablement rather than blocking flow.
- ☐ Work does not pause while waiting for another functional silo.
- ☐ Operational responsibility stays close to the teams building the software.
- ☐ Support teams, if needed, are aligned to streams rather than centralized around all systems.

Converting Existing Team Types

Infrastructure Teams

- ☐ Infrastructure teams are being moved toward platform-team behavior.
- ☐ Infrastructure services are offered as self-service capabilities.
- ☐ Infrastructure work reduces cognitive load for stream-aligned teams.
- ☐ Infrastructure teams avoid owning application-level changes for other teams.

Component Teams

- ☐ Component teams are reviewed to determine whether they should be dissolved, converted to platform teams, enabling teams, or complicated-subsystem teams.
- ☐ Component teams do not exist merely because of technical architecture.
- ☐ Component ownership does not create unnecessary hand-offs.

Tooling Teams

- ☐ Tooling teams have a clear, time-bound, or product-oriented mission.
- ☐ Tooling teams do not become isolated tool-maintenance teams.
- ☐ Tooling teams are converted into enabling teams or platform teams where appropriate.

Support Teams

- ☐ Support teams are aligned to streams of change where possible.
- ☐ Support activity helps teams learn from incidents.
- ☐ Support does not centralize all production responsibility away from stream-aligned teams.
- ☐ Dynamic swarming is used for incidents that affect multiple streams.

Architecture Teams

- ☐ Architecture is handled through part-time enabling where possible.

- ☐ Architects support teams rather than impose decisions.
- ☐ Architecture work helps shape APIs, boundaries, and team interactions.
- ☐ Architecture decisions consider Conway's Law and team communication paths.

Final Health Check

- ☐ Most teams are stream-aligned.
- ☐ Supporting teams reduce cognitive load.
- ☐ Enabling teams increase capability and then move on.
- ☐ Complicated-subsystem teams exist only where specialist knowledge is truly required.
- ☐ Platform teams provide useful internal products.
- ☐ Teams are optimized for fast, safe, sustainable flow of change.
- ☐ Team boundaries are clear, purposeful, and easy to explain.